

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра Информационных систем

ПРАКТИЧЕСКАЯ РАБОТА № 1
по дисциплине «Методы и средства проектирования
информационных систем»
ТЕМА: ПК и СЕРВЕРА

Группа	ФИО	Подпись исп.	Оценка	Подпись преп.
3376	Михайлов Н.Ф.			
3376	Константинов Р.И.			
3376	Дегтярева М.И.			

Преподаватель

Дюков Н.В.

Санкт-Петербург

2026

ЗАДАНИЕ

НА ПРАКТИЧЕСКУЮ РАБОТУ №1

Тема работы: ПК и Сервера

Исходные данные:

Создать классификатор изделий, который хранится в базе данных, и сервер, позволяющий получать доступ к БД посредством сервера API.

Содержание пояснительной записки:

«Аннотация», «Содержание», «Введение», «Анализ предметной области», «Разработка функциональных требований», «Разработка проектной модели», «Разработка модели хранения», «Разработка базы данных», «Разработка серверной части», «Заключение», «Список использованных источников», «Приложение».

Предполагаемый объем пояснительной записки:

Не менее 15 страниц.

Дата выдачи задания: 06.03.2026

Дата сдачи реферата:

Дата защиты реферата:

Преподаватель

Михайлов Н.Ф.

Константинов Р.И.

Дегтярева М.И.

Дюков Н.В.

АННОТАЦИЯ

В данной работе представлена разработка классификатора изделий магазина ПК и серверов для удобного хранения каталога комплектующих ПК и серверов, а также навигации по нему с помощью базы данных и API.

Для выполнения задачи нужно: изучить предметную область, определить функциональные требования через use-case диаграмму, создать диаграмму классов (схему классификатора), а также спроектировать базу данных с помощью ER-диаграммы. Основной акцент сделан на построении базы данных с возможностью динамического изменения ассортимента комплектующих и разработке серверной логики для поиска, классификации и навигации в рамках магазина ПК и серверов.

SUMMARY

This paper presents the development of a product classifier for a PC and server store, designed to facilitate the storage of a catalog of PC and server components and navigation within it using a database and an API.

To accomplish this task, it is necessary to: study the subject domain, define functional requirements using a use-case diagram, create a class diagram (classifier schema), and design a database using an ER diagram. The main focus is on building a database that allows for dynamic changes to the range of components and developing server-side logic for search, classification, and navigation within the PC and server store.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
2. РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ	8
3. РАЗРАБОТКА ПРОЕКТНОЙ МОДЕЛИ	10
4. РАЗРАБОТКА МОДЕЛИ ХРАНЕНИЯ	11
5. РАЗРАБОТКА БАЗЫ ДАННЫХ	13
6. РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ	18
ЗАКЛЮЧЕНИЕ	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30
ПРИЛОЖЕНИЕ А	31

ВВЕДЕНИЕ

В современной IT-индустрии каталогизация ассортимента ПК и серверов играет ключевую роль в эффективном управлении магазином. Предприятиям необходимо отслеживать наличие товаров, их характеристики, планировать закупки и продажи, оптимизировать склады, формировать рекомендации для пользователей.

Классификатор ассортимента ПК и серверов позволяет видеть данные в иерархической структуре - от общих категорий до конкретных моделей с описанием характеристик, со статусом наличия и ценой. Такой подход упрощает управление большим ассортиментом, позволяет быстро адаптировать данные в каталоге при необходимых изменениях, формировать учет на всех этапах.

Целью данной работы является проектирование и разработка гибкой информационной системы для классификатора ассортимента ПК, серверов и комплектующих магазина. Система на основе базы данных и API-сервера позволит динамически изменять структуру классификатора без правок в коде, обеспечивая операции добавления, удаления и редактирования товаров.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Объектом автоматизации является процесс учёта и классификации ПК, сервером и их комплектующих. Предметная область характеризуется иерархической структурой комплектующих, которую необходимо представить в виде классификатора, позволяющего организовать ассортимент по типу комплектующего, подкатегориям и итоговым элементам.

Для наглядного представления фрагмента классификатора, покажем его структуру данных на рисунке 1

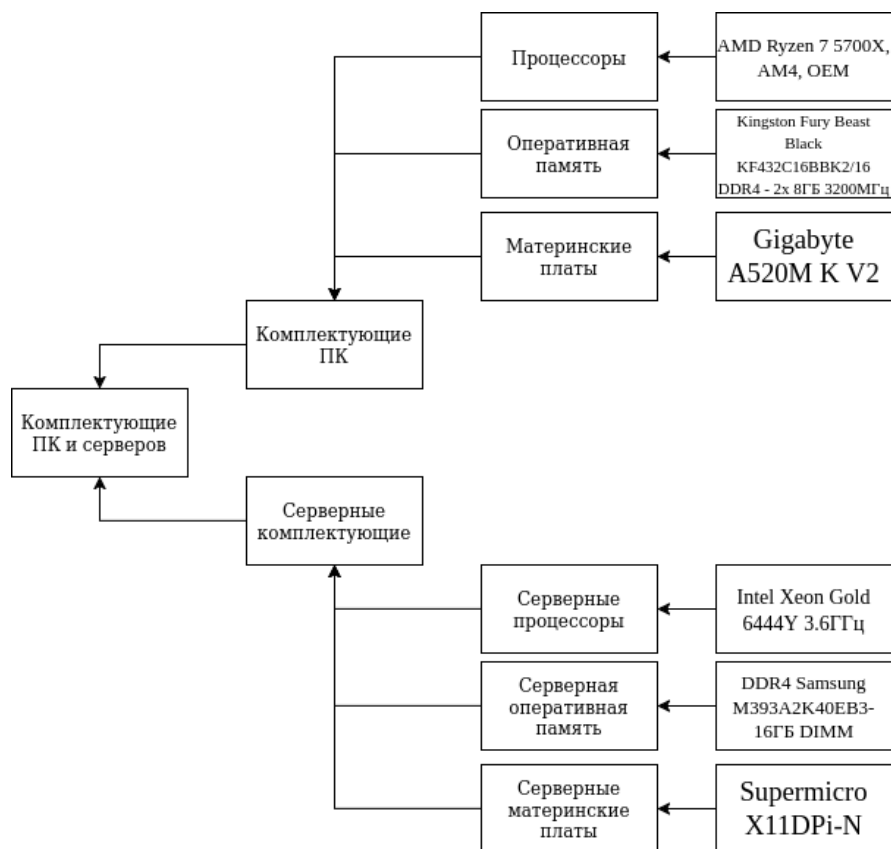


Рисунок 1 — Структура данных

Рассмотрим

структуру

данной

системы:

Комплектующие ПК и сервера — корень, объединяющий все комплектующие.

Промежуточные классы (узлы) — категория, используемая для категоризации комплектующих

Терминальные классы (листья) — конечные элементы, представляющие собой конкретные комплектующие. Не имеют потомков.

1. Комплектующие ПК и сервера — корень, объединяющий все комплектующие.

1.1. Комплектующие ПК

1.1.1. Процессоры

1.1.1.1. “AMD Ryzen 7 5700X, AM4, OEM”

1.1.2. Оперативная память

1.1.2.1. “Kingston Fury Beast Black
KF432C16BBK2/16 DDR4 - 2x 8ГБ
3200МГц”

1.1.3. Материнские платы

1.1.3.1. “Gigabyte A520M K V2”

1.2. Серверные комплектующие

1.2.1. Серверные процессоры

1.2.1.1. “Intel Xeon Gold 6444Y 3.6ГГц”

1.2.2. Серверная оперативная память

1.2.2.1. “DDR4 Samsung M393A2K40EB3-16ГБ
DIMM”

1.2.3. Серверные материнские платы

1.2.3.1. “Supermicro X11DPi-N”

Данный фрагмент представляет собой пример структуры данных, который может быть расширена новыми подкатегориями и конкретными примерами комплектующих.

2. РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

Для демонстрации принципа работы системы была разработана диаграмма прецедентов (Рисунок 2), на которой показаны основные участники системы и методы их взаимодействия с разрабатываемой моделью классификатора.

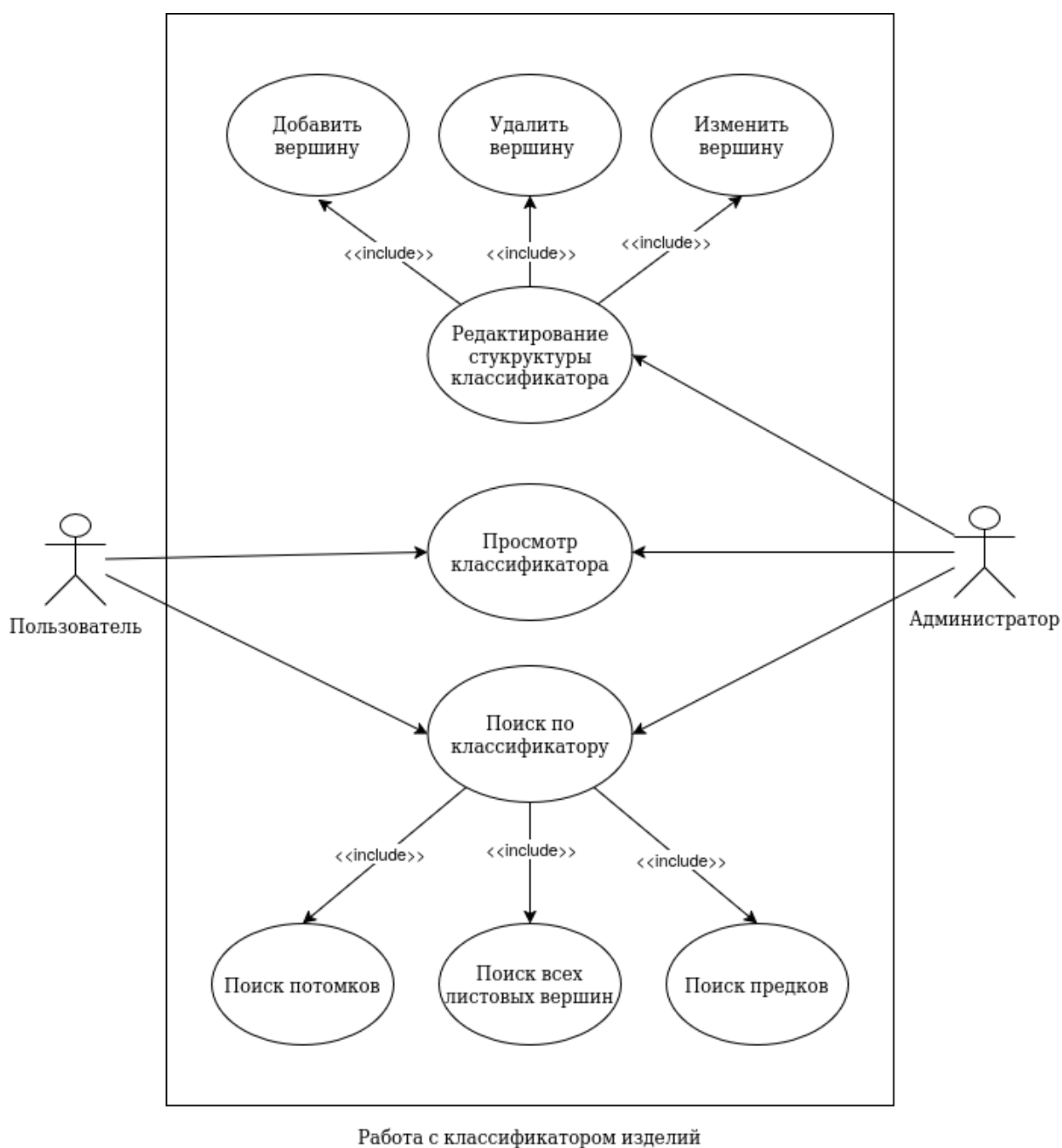


Рисунок 2 — Диаграмма прецедентов

На диаграмме представлены следующие акторы:

- Пользователь — клиент, просматривающий содержимое классификатора, также осуществляющий поиск по этому классификатору.
- Администратор — Пользователь, способный редактировать структуру классификатора путём добавления / редактирования / удаления узлов классификатора .

На диаграмме представлены следующие сценарии поведения акторов:

Сценарии поведения пользователя:

- Просмотр классификатора — получение полной структуры ПК / Серверных комплектующих.
- Поиск потомков — получение списка подкатегорий и продуктов для выбранного узла.
- Поиск предков — получение списка родительских категорий выбранной подкатегории или продукта.
- Поиск всех листовых вершин — получение продуктов для данной категории.

Сценарии поведения администратора:

- Все описанные сценарии пользователя.
- Добавление вершины.
- Удаление вершины.
- Изменение вершины.

3. РАЗРАБОТКА ПРОЕКТНОЙ МОДЕЛИ

Для наглядного представления работы системы была составлена диаграмма классов, представленная на рисунке 3. Диаграмма отображает ключевые атрибуты и методы классов, а также соотношение между ними.

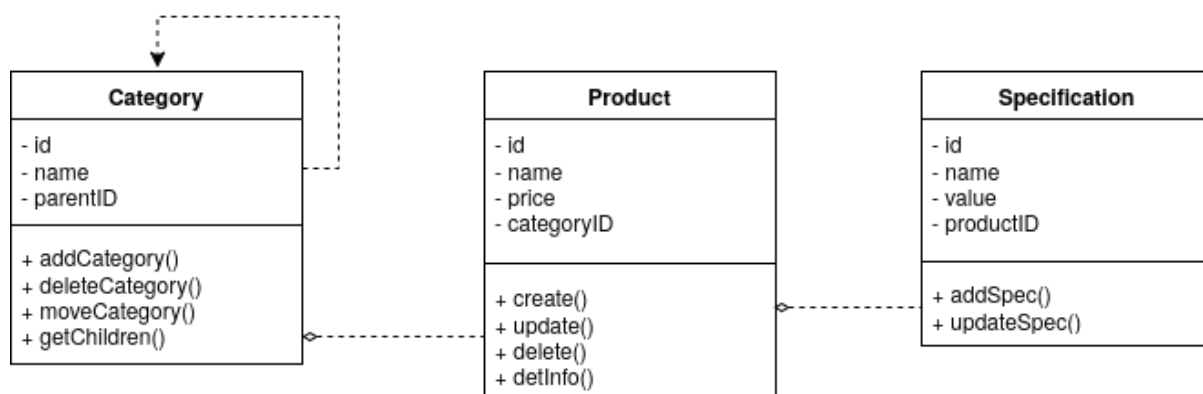


Рисунок 3 — Диаграмма классов

Класс Category – это метакласс, который представляет собой категорию комплектующих для ПК. Он используется для организации структуры каталога (классификатора), позволяет структурировать и категоризировать комплектующие. Из таких категорий строится иерархическое дерево.

Класс Product – это класс, олицетворяющий конкретные комплектующие, используя такие атрибуты как: название, ID категории, которой он принадлежит, цена и ID самого товара.

Класс Specification – это класс, который содержит разные единицы измерения. С их помощью идёт описание характеристик конкретных комплектующих.

4. РАЗРАБОТКА МОДЕЛИ ХРАНЕНИЯ

Для хранения информации о продукции используется реляционная модель данных, представленная на рисунке 4, которая включает в себя названия таблиц.

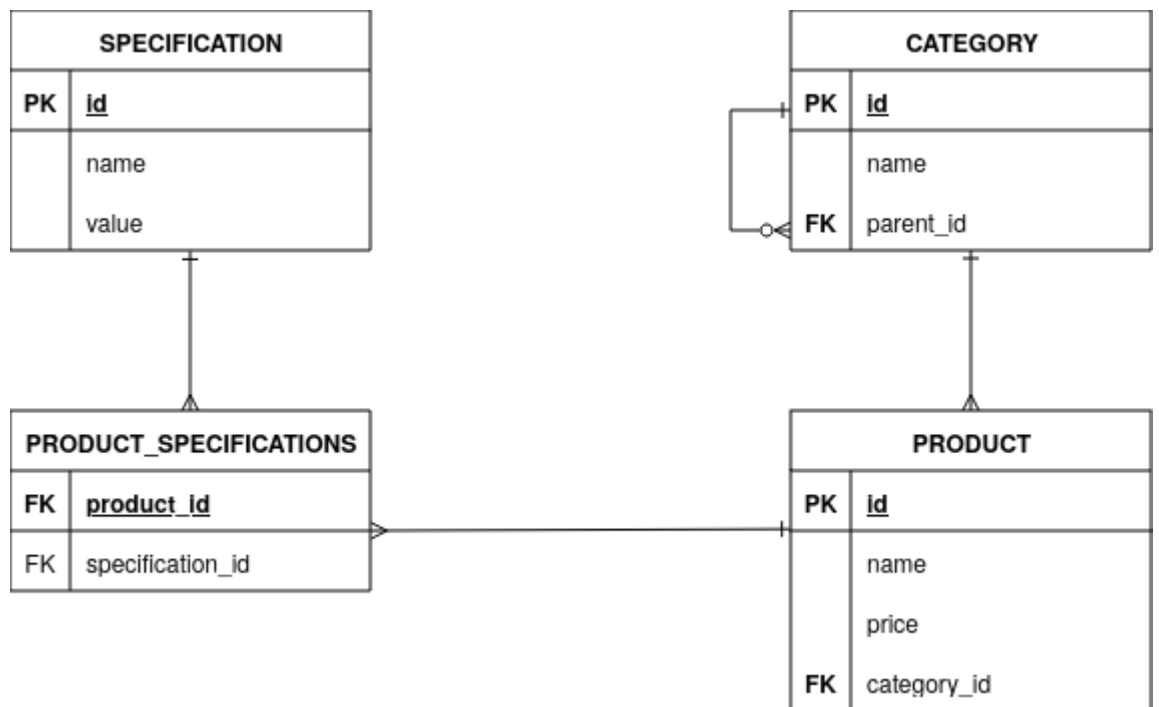


Рисунок 4 — ER диаграмма

SPECIFICATION:

- **id (PK)** — Уникальный идентификатор спецификации. Первичный ключ для каждой записи.
- **name** — Название атрибута.
- **value** — Значение атрибута.

PRODUCT_SPECIFICATION:

- **product_id** — идентификатор продукта.
- **specification_id** — идентификатор спецификации.

PRODUCT:

- id (PK) — Уникальный идентификатор продукта.
- name — Название продукта.
- price — Цена продукта в валюте.
- category_id (FK) — Ссылка на категорию продукта.

CATEGORY:

- id (PK) — Уникальный идентификатор продукта.
- name — Название категории.
- parent_id (FK) — Ссылка на родительскую категорию для дерева.

5. РАЗРАБОТКА БАЗЫ ДАННЫХ

Для хранения данных разрабатываемого классификатора была выбрана реляционная база данных PostgreSQL, так как она обеспечивает надежное хранение структурированной информации, поддерживает механизм внешних ключей и позволяет эффективно работать со связями между сущностями. А так же у команды разработки уже был опыт работы с ней.

Запуск базы данных осуществляется на локальном сервере PostgreSQL с использованием docker compose. При старте приложения автоматически выполняется создание базы данных, создание таблиц и подготовка корневой вершины классификатора.

Рассмотрим в таблице 1 основные функции базы данных, обеспечивающие создание структуры хранения, добавление, изменение, удаление и перемещение элементов классификатора, а также управление продуктами и спецификациями. Примеры кода некоторых реализованных функций приведены в приложении А.

Таблица 1 - Функции базы данных

Название функции	Назначение	Вход	Выход
create_tables	Создание таблиц базы данных и подготовка структуры хранения	отсутствует	созданная схема БД
ensure_root_category	Создание или получение корневой категории классификатора	отсутствует	id корневой категории

Продолжение таблицы 1 - Функции базы данных

Название функции	Назначение	Вход	Выход
create_category	Добавление новой вершины классификатора	name - название категории, parent_id - идентификатор родительской вершины	новый объект категории
update_category	Изменение названия существующей категории	category_id, name	обновленный объект категории
move_category	Перемещение категории в другую часть дерева	category_id, new_parent_id	обновленный объект категории
delete_category	Удаление категории при отсутствии дочерних категорий и товаров	category_id	подтверждение удаления
create_product	Добавление нового комплектующего в выбранную категорию	category_id, name, price, quantity, description, specifications	новый объект товара

Продолжение таблицы 1 - Функции базы данных

Название функции	Назначение	Вход	Выход
update_product	Изменение данных существующего комплектующего	product_id, name, price, quantity, description, specifications	обновленный объект товара
move_product	Перемещение комплектующего в другую категорию	product_id, new_category_id	обновленный объект товара
delete_product	Удаление комплектующего	product_id	подтверждение удаления
resolve_specification_references	Поиск существующих спецификаций или подготовка новых к сохранению	список спецификаций	список разрешенных спецификаций
_get_or_create_cannonical_specification	Создание новой спецификации либо получение уже существующей	`name`, `value` объект спецификации	объект спецификации

Окончание таблицы 1 - Функции базы данных

Название функции	Назначение	Вход	Выход
list_specifications	Получение полного справочника характеристик	отсутствует	список спецификаций
get_tree	Построение полного дерева классификатора вместе с товарами и характеристикам и	отсутствует	объект дерева классификатора
get_database_summary	Получение краткой информации о состоянии базы данных	отсутствует	статистическая информация по БД

Пример заполнения таблицы представлен на рисунке 5.

id	name	category_id	price	quantity	description
1	AMD Ryzen 7 5700X	7	14,990	7	8 ядер, 16 потоков
2	Kingston Fury Beast Black KF432C16BBK2/16 DDR4 - 2x 8ГБ 3200МГц	8	6,990	12	Оперативная память для ПК
3	Gigabyte A520M K V2	9	7,990	5	Материнская плата для ПК
4	Intel Xeon Gold 6444Y 3.6ГГц	6	159,990	2	Серверный процессор
5	DDR4 Samsung M393A2K40EB3-16ГБ DIMM	5	11,990	10	Серверная оперативная память
6	Supermicro X11DPI-N	4	45,990	3	Серверная материнская плата

Рисунок 5 – пример таблицы products с тестовыми данными

Пример заполнения таблицы categories необходимыми категориями представлен на рисунке 6.

123 id	AZ name	123 parent_id
1	Комплектующие ПК и серверов	[NULL]
2	Комплектующие для ПК	1
7	Процессоры для ПК	2
8	Оперативная память для ПК	2
4	Серверные материнские платы	3
3	Серверные комплектующие	1
5	Серверная оперативная память	3
6	Серверные процессоры	3
9	Материнские платы для ПК	2

Рисунок 6 – пример таблицы categories с тестовыми данными

Функции, представленные выше в таблице 1, будут рассмотрены в ходе представления работы серверной части программы.

6. РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ

Серверная часть приложения реализована на языке программирования Python с использованием фреймворка FastAPI. Выбор данного фреймворка обусловлен его удобством при создании REST API, поддержкой автоматической генерации документации и возможностью быстрого развертывания серверного приложения.

Разработанный сервер обеспечивает доступ к базе данных классификатора комплектующих ПК и серверов посредством HTTP-запросов. Через сервер реализуются операции создания, изменения, удаления и перемещения категорий классификатора, а также операции работы с конечными комплектующими и их спецификациями.

Приложение развернуто с использованием Docker и описано в файле конфигурации docker-compose.yml.

Система состоит из двух контейнеров:

- контейнера базы данных PostgreSQL
- контейнера серверного приложения FastAPI

Для взаимодействия с серверной частью используется REST API. Дополнительно FastAPI автоматически предоставляет Swagger UI, что упрощает тестирование и демонстрацию работы системы.

В таблице 2 рассмотрим реализованные методы подробнее.

Таблица 2 - Реализованные методы HTTP

Метод	URL	Описание
GET	/info/ping	Проверка доступности серверного приложения

Продолжение таблицы 2 - Реализованные методы HTTP

Метод	URL	Описание
GET	/database/info	Получить краткую информацию о состоянии базы данных
POST	/database/tables/create	Создать таблицы базы данных и подготовить структуру хранения
GET	/database/tree	Получить полное дерево классификатора вместе с конечными комплектующими
GET	/database/specifications	Получить справочник уникальных спецификаций
POST	/database/categories	Создать новую вершину классификатора
PATCH	/database/categories/{category_id}	Изменить название вершины классификатора
PATCH	/database/categories/{category_id}/move	Переместить вершину в другую ветку дерева

Окончание таблицы 2 - Реализованные методы HTTP

Метод	URL	Описание
DELETE	/database/categories/{category_id}	Удалить вершину классификатора
POST	/database/products	Добавить новый продукт в выбранную категорию
PATCH	/database/products/{product_id}	Обновить данные продукта
PATCH	/database/products/{product_id}/move	Переместить продукт в другую категорию
DELETE	/database/products/{product_id}	Удалить продукт

Проведем тестирование разработанной серверной части. Для начала осуществляется запуск приложения с использованием Docker Compose. Пример запуска представлен на рисунке 7.

```

$ docker compose up
Attaching to mispris-api, mispris-postgres
Container mispris-postgres is restarting
mispris-postgres | PostgreSQL Database directory appears to contain a database; Skipping initialization
mispris-postgres | 2026-04-07 21:56:23.572 UTC [1] LOG: starting PostgreSQL 16.12 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
mispris-postgres | 2026-04-07 21:56:23.572 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
mispris-postgres | 2026-04-07 21:56:23.572 UTC [1] LOG: listening on IPv6 address ":::", port 5432
mispris-postgres | 2026-04-07 21:56:23.574 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
mispris-postgres | 2026-04-07 21:56:23.579 UTC [29] LOG: database system was shut down at 2026-04-07 21:56:17 UTC
mispris-postgres | 2026-04-07 21:56:23.583 UTC [1] LOG: database system is ready to accept connections
Container mispris-postgres is healthy
mispris-api | INFO: Started server process [1]
mispris-api | INFO: Waiting for application startup.
mispris-api | INFO: Application startup complete.
mispris-api | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)

```

Рисунок 7 - Запуск сервера

Далее через метод `POST /database/categories` возможно добавление новых узлов дерева, например категорий "Комплектующие для ПК", "Серверные

комплектующие", "Процессоры для ПК", "Серверные процессоры" и других разделов каталога (Рисунок 8).

The screenshot shows a REST client interface with the following sections:

- POST /database/categories** (Create category)
- Parameters:** No parameters.
- Request body:** Required, set to `application/json`. The JSON body is:

```
{  "name": "Корпуса для ПК",  "parent_id": 2}
```
- Execute:** A blue button to execute the request.
- Responses:**
 - Curl:**

```
curl -X 'POST' \  "http://127.0.0.1:8000/database/categories" \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "name": "Корпуса для ПК",  "parent_id": 2}'
```
 - Request URL:** `http://127.0.0.1:8000/database/categories`
 - Server response:**

Code	Details
201	Response body <pre>{ "id": 10, "name": "Корпуса для ПК", "parent_id": 2}</pre> Response headers <pre>content-length: 59 content-type: application/json date: Tue, 07 Apr 2026 21:58:21 GMT server: uvicorn</pre>

Рисунок 8 - Добавление нового узла классификатора

После создания категорий можно выполнить запрос `GET /database/tree` и получить полную структуру классификатора вместе со всеми дочерними категориями. Это позволяет проверить корректность добавления узлов и их привязку к родительским вершинам (Рисунок 9).

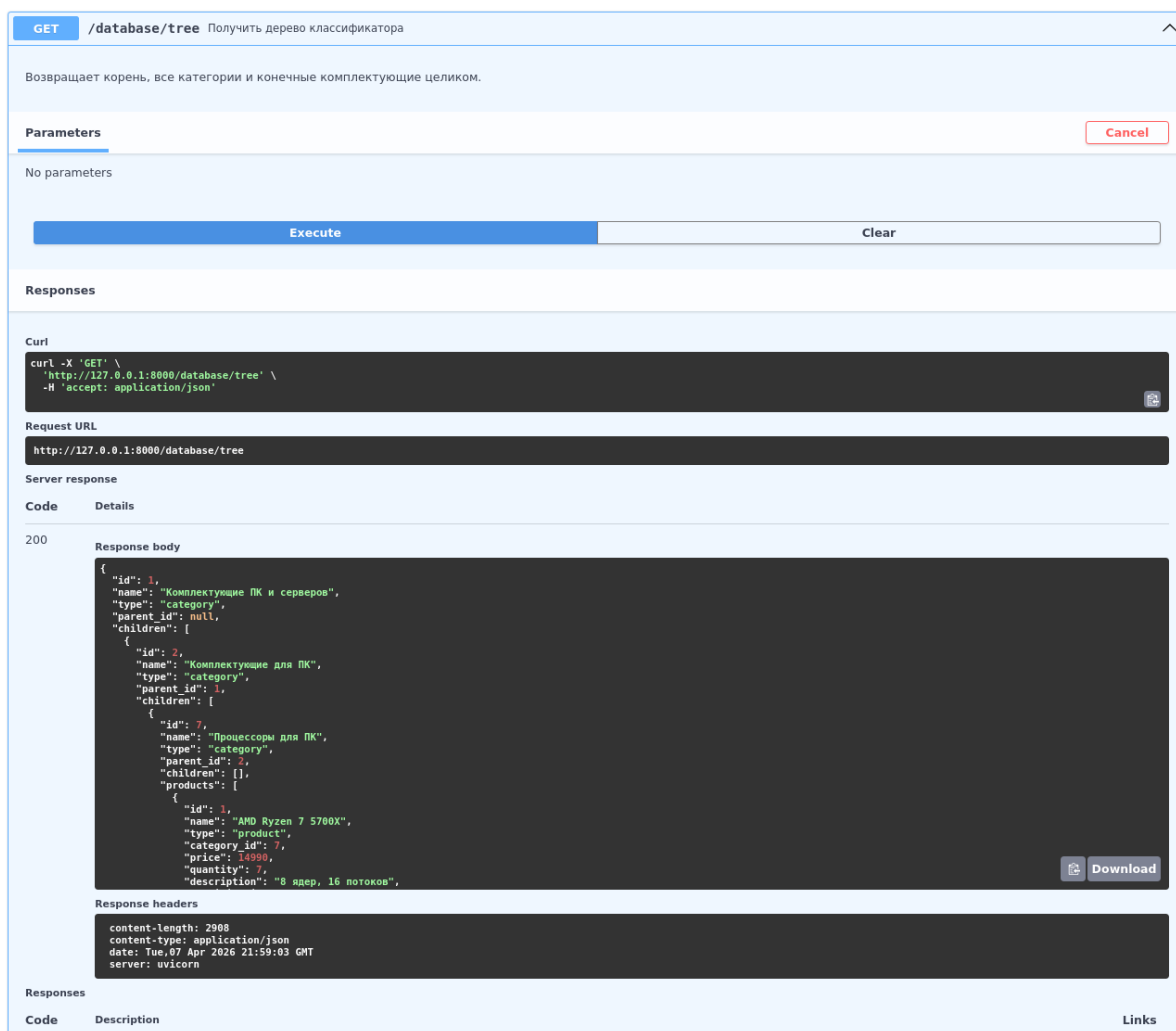


Рисунок 9 - Получение дерева классификатора

Следующим этапом выполняется добавление конечных комплектующих. Для этого используется метод `POST /database/products`, который позволяет создать новый товар в выбранной категории и одновременно задать набор характеристик. Если указанная характеристика уже существует в справочнике, сервер использует существующую запись; если не существует, она создается автоматически (Рисунок 10).

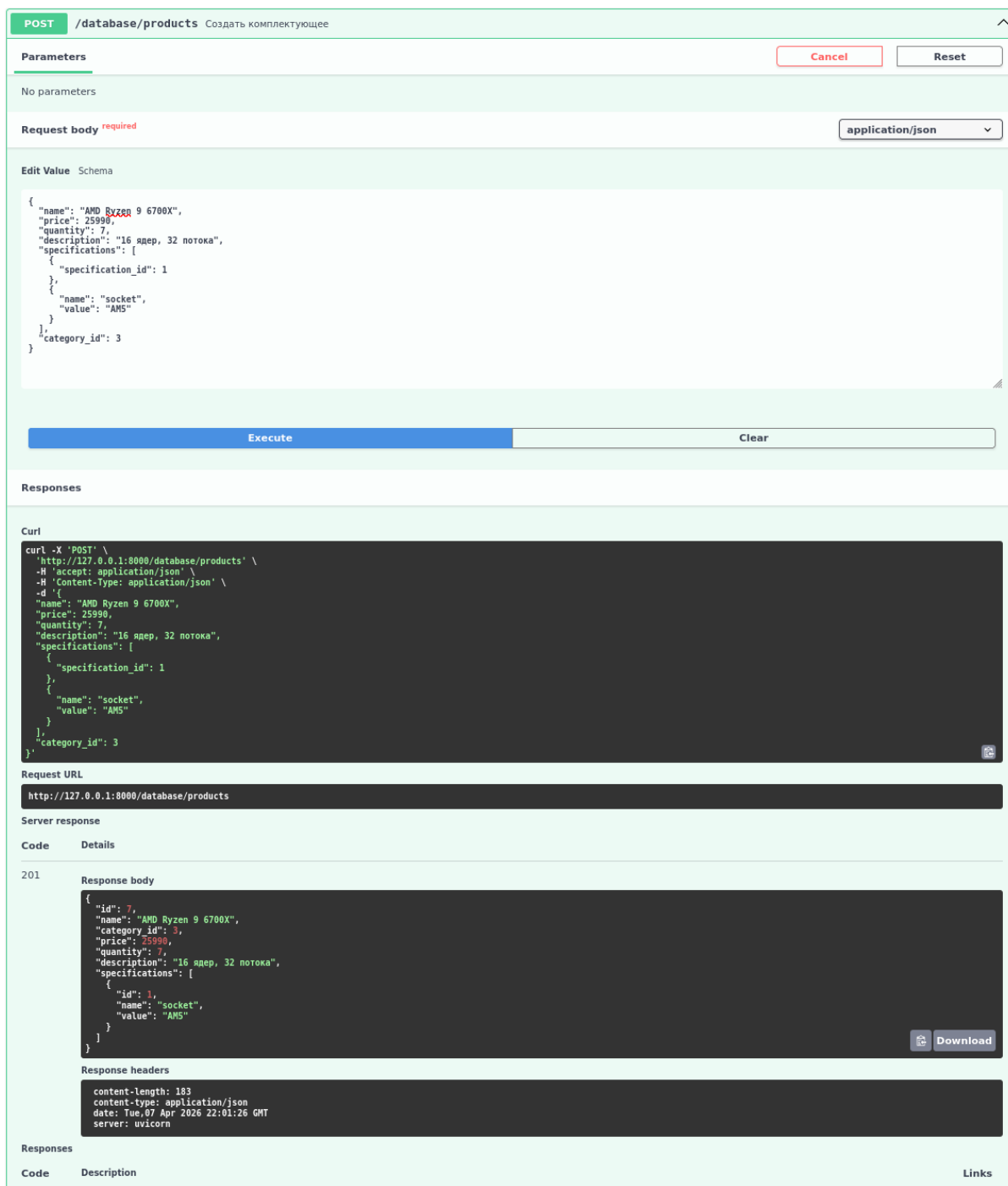


Рисунок 10 - Добавление нового продукта

Для проверки корректности работы механизма спецификаций используется метод `GET /database/specifications`, который выводит справочник всех уникальных характеристик, добавленных в систему (Рисунок 11).

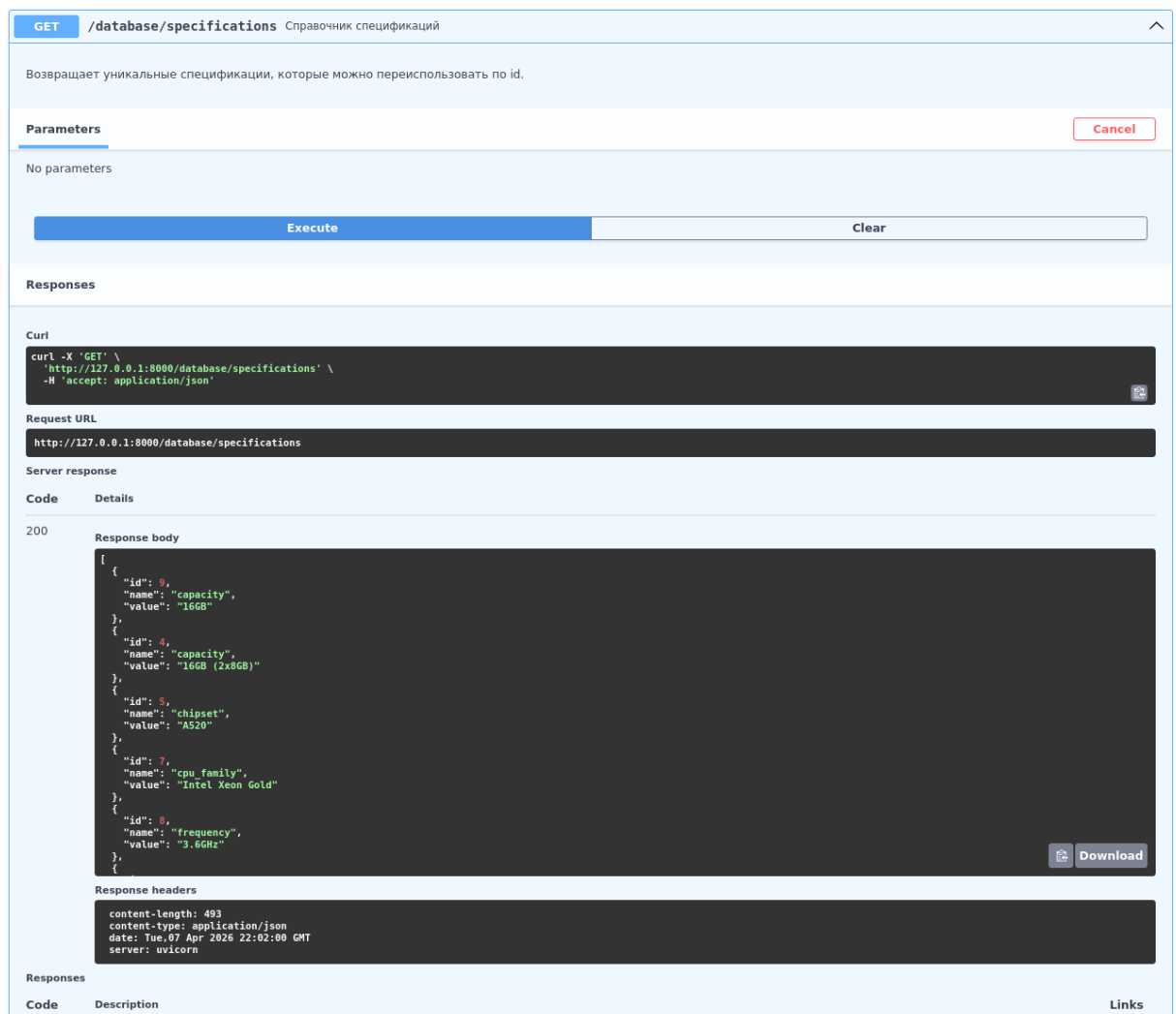


Рисунок 11 - Получение справочника спецификаций

Рассмотрим изменение структуры дерева. Для этого может быть использован метод `PATCH /database/categories/{category\_id}/move`, который позволяет изменить положение выбранной категории внутри классификатора. После выполнения данной операции можно повторно получить дерево через `GET /database/tree` и убедиться, что категория была перемещена в новую ветку (Рисунок 12).

PATCH
/database/categories/{category_id}/move
Переместить категорию

Cancel

Reset

Parameters

Name	Description
category_id * required integer (path)	10

Request body required

application/json

Edit Value | Schema

```
{
  "new_parent_id": 3
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PATCH' \
  'http://127.0.0.1:8080/database/categories/10/move' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "new_parent_id": 3
  }'
```

Request URL

```
http://127.0.0.1:8080/database/categories/10/move
```

Server response

Code	Details
200	Response body <pre>{ "id": 10, "name": "Копюса для ПК", "parent_id": 3 }</pre> Download Response headers <pre>content-length: 59 content-type: application/json date: Tue-07 Apr 2026 22:03:39 GMT server: uvicorn</pre>

Responses

Code	Description
------	-------------

Links

Рисунок 12 - Перемещение вершины классификатора

Аналогично возможно изменение данных существующей категории или товара. Для категории используется метод `PATCH /database/categories/{category\_id}`, а для продукта - `PATCH /database/products/{product\_id}`. Это позволяет изменять названия, описания, цены, количество и набор характеристик без удаления существующих записей (Рисунок 13).

PATCH /database/products/{product_id} Изменить комплектующее

Parameters

Cancel

Reset

Name	Description
product_id required	1
integer	
(path)	

Request body required

application/json

Edit Value

Schema

```

{
  "name": "AMD Ryzen 7 5700X",
  "price": 14990,
  "quantity": 7,
  "description": "16 ядер, 16 потоков",
  "specifications": [
    {
      "specification_id": 1
    }
  ],
  "name": "socket",
  "value": "AMS"
}

```

Execute

Clear

Responses

Curl

```

curl -X 'PATCH' \
  'http://127.0.0.1:8000/database/products/1' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "AMD Ryzen 7 5700X",
    "price": 14990,
    "quantity": 7,
    "description": "16 ядер, 16 потоков",
    "specifications": [
      {
        "specification_id": 1
      }
    ],
    "name": "socket",
    "value": "AMS"
  }'

```

Request URL

http://127.0.0.1:8000/database/products/1

Server response

Code	Details
200	Response body <pre> { "id": 1, "name": "AMD Ryzen 7 5700X", "category id": 7, "price": 14990, "quantity": 7, "description": "16 ядер, 16 потоков", "specifications": [{ "id": 1, "name": "socket", "value": "AMS" }] } </pre> Download Response headers <pre> content-length: 185 content-type: application/json date: Tue, 07 Apr 2026 22:07:10 GMT server: uvicorn </pre>

Рисунок 13 - Изменение данных категории или продукта

Если внесенные изменения больше не требуются, система позволяет удалить объект с помощью метода `DELETE`. Для категорий реализована дополнительная проверка: вершина не может быть удалена, если в ней содержатся дочерние категории или товары. Такая проверка предотвращает нарушение целостности классификатора (Рисунок 14).

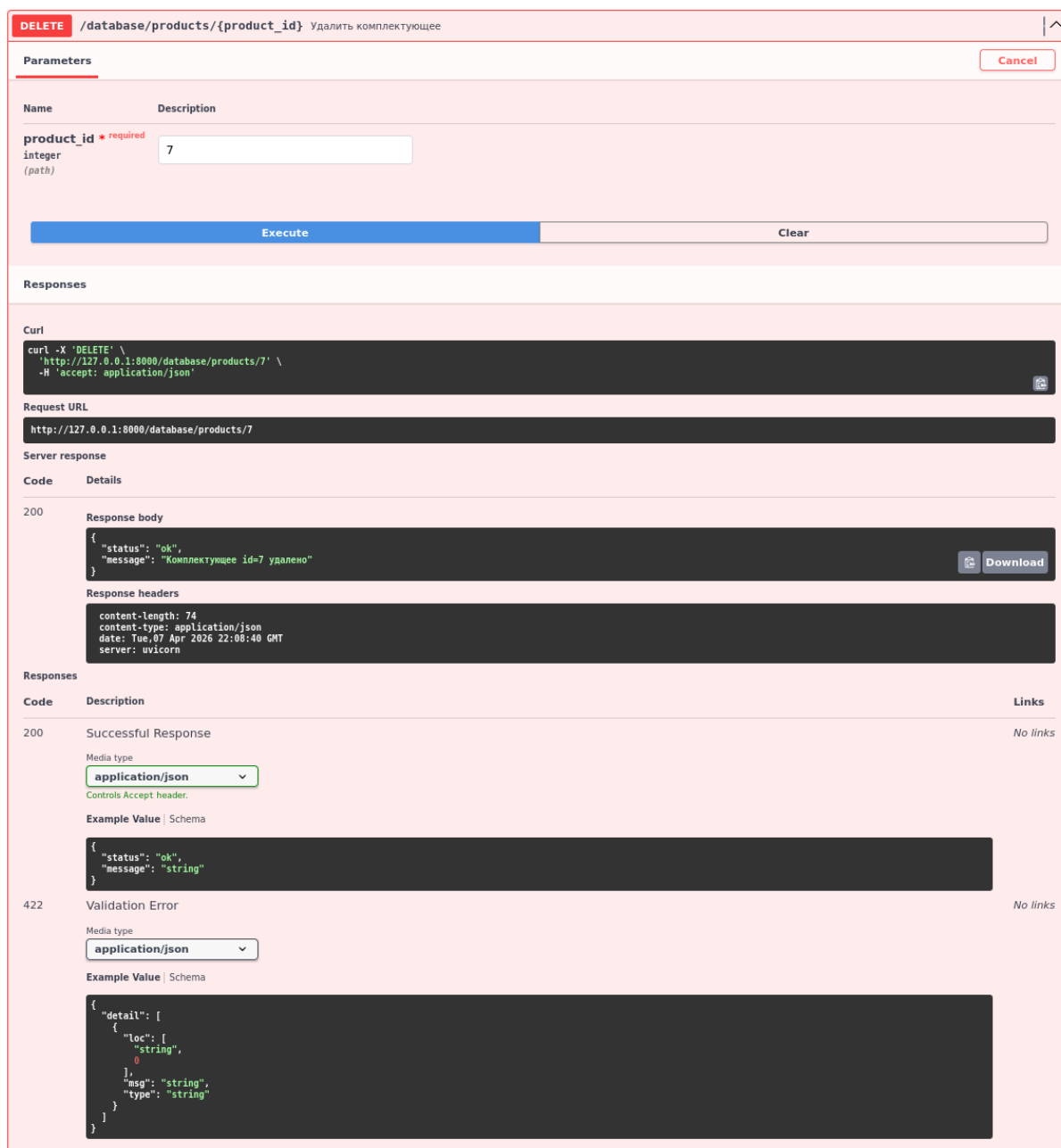


Рисунок 14 - Удаление объекта классификатора

После выполнения операций удаления или перемещения можно повторно выполнить запрос на получение дерева и убедиться, что структура данных изменилась корректно и соответствует ожидаемому состоянию (Рисунок 15).

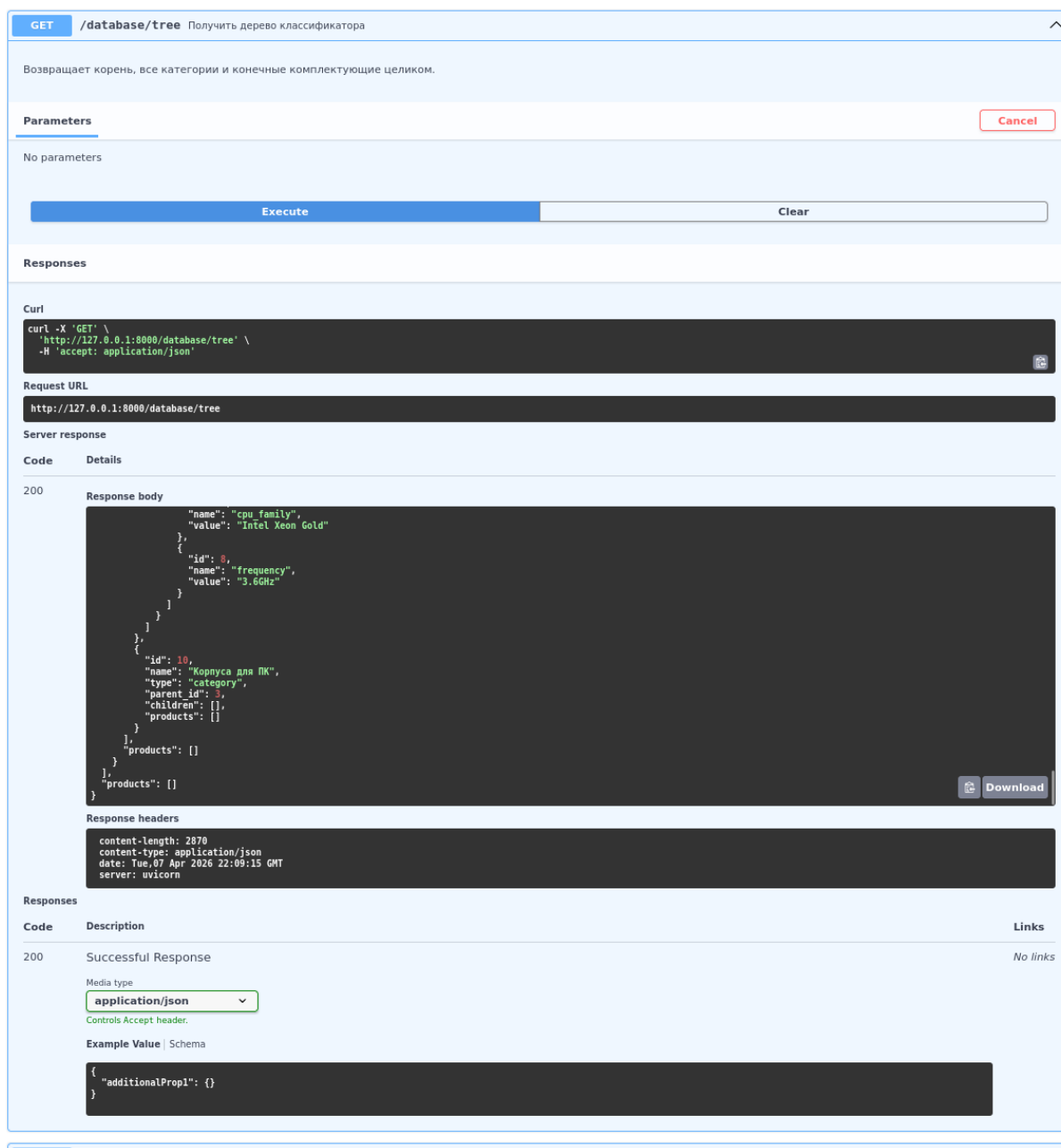


Рисунок 15 - Проверка результата после выполнения операций

Таким образом, по результатам тестирования можно сделать вывод, что серверная часть приложения корректно реализует все основные функции взаимодействия с классификатором. Реализованные HTTP-методы обеспечивают управление категориями, товарами и спецификациями, поддерживают работу с полной древовидной структурой и позволяют выполнять демонстрацию системы через Swagger UI. Это подтверждает корректность реализации серверной части и ее соответствие поставленным требованиям.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы была разработана система классификации продукции на примере ассортимента комплектующих для ПК и серверов, включающая базу данных и серверную часть приложения. Реализованный классификатор позволяет гибко управлять иерархией категорий, конечными комплектующими и их характеристиками без изменения программного кода.

Поставленная цель была достигнута: разработана структура базы данных, реализованы основные операции для работы с категориями, товарами и спецификациями, а также обеспечено взаимодействие с системой через серверную часть. Результаты тестирования показали корректность работы приложения и его соответствие поставленным требованиям.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ Р 7.0.97–2025. Национальный стандарт Российской Федерации. Система стандартов по информации, библиотечному и издательскому делу. Организационно-распорядительная документация. Требования к оформлению документов. М.: Стандарт, 2025.
2. Каталог интернет-магазина техники // Ситилинк URL: <https://www.citilink.ru/catalog/komplektuyushchie-dlya-pk/?ref=mainmenu> (дата обращения: 01.04.2026).
3. Каталог интернет-магазина техники // DNS URL: <https://www.dns-shop.ru/catalog/17aa529516404e77/komplektuyushchie-dla-servera/> (дата обращения 01.04.2026).

ПРИЛОЖЕНИЕ А

```
async def create_tables() -> None:
    engine = get_engine()
    async with engine.begin() as conn:
        await conn.run_sync(metadata.create_all)

    async with get_session_factory()() as session:
        await ensure_root_category(session)
        await prepare_specifications_schema(session)
        await migrate_legacy_specifications(session)
        await session.commit()

async def create_category(name: str, parent_id: int | None = None) -> dict[str,
object]:
    async with get_session_factory()() as session:
        root_id = await ensure_root_category(session)
        resolved_parent_id = parent_id if parent_id is not None else root_id

        parent = await _get_category_row(session, resolved_parent_id)
        if parent is None:
            raise TreeError(f"Нельзя создать вершину: родитель с
id={resolved_parent_id} не найден")

        if await _category_exists_with_name(
            session,
            parent_id=resolved_parent_id,
            name=name,
        ):
            raise TreeError(
                f"Нельзя создать вершину: в родительской категории
id={resolved_parent_id} уже есть элемент с именем '{name}'"
            )

        result = await session.execute(
            insert(categories_table)
            .values(name=name.strip(), parent_id=resolved_parent_id)
            .returning(categories_table)
        )
        await session.commit()
        row = result.mappings().one()
        return {
            "id": int(row.id),
```

```

        "name": str(row.name),
        "parent_id": int(row.parent_id) if row.parent_id is not None else None,
    }

async def delete_category(category_id: int) -> None:
    async with get_session_factory()() as session:
        category = await _get_category_row(session, category_id)
        if category is None:
            raise TreeError(f"Нельзя удалить вершину: категория с id={category_id} не найдена")

        has_children = await session.scalar(
            select(categories_table.c.id).where(categories_table.c.parent_id ==
category_id)
        )
        if has_children is not None:
            raise TreeError("Нельзя удалить вершину: у нее есть дочерние категории")

        has_products = await session.scalar(
            select(products_table.c.id).where(products_table.c.category_id ==
category_id)
        )
        if has_products is not None:
            raise TreeError("Нельзя удалить вершину: в ней есть комплектующие")

        await session.execute(
            delete(categories_table).where(categories_table.c.id == category_id)
        )
        await session.commit()

async def create_product(
    *,
    category_id: int,
    name: str,
    price: Decimal,
    quantity: int,
    description: str | None,
    specifications: list[dict[str, object]],
) -> dict[str, object]:
    async with get_session_factory()() as session:
        result = await session.execute(

```



```

insert(products_table)
    .values(
        category_id=category_id,
        name=name.strip(),
        price=price,
        quantity=quantity,
        description=description.strip() if description else None,
    )
    .returning(products_table)
)
product_row = result.mappings().one()
product_id = int(product_row["id"])

resolved_specs = await resolve_specification_references(session,
specifications)
for spec in resolved_specs:
    await session.execute(
        insert(product_specifications_table).values(
            product_id=product_id,
            specification_id=int(spec["id"]),
        )
    )

await session.commit()
return await get_product(product_id)

```